CREACIÓN DE LLMs

Guía Técnica Completa

NIVEL 2	NIVEL 3
Fine-Tuning	Transformer desde Cero
Especializar un modelo existente	Implementar la arquitectura completa

Asignatura: Agentes Inteligentes | Ingeniería de Software | 2025

1. Introducción

Este documento presenta una guía técnica completa para crear Modelos de Lenguaje de Gran Escala (LLMs) en dos niveles de complejidad: Fine-Tuning de modelos preentrenados (Nivel 2) e implementación del Transformer desde cero en PyTorch (Nivel 3).

NOTA	El Nivel 1 (entrenar un LLM completo como GPT-4 o Claude) requiere decenas de millones de dólares en hardware, terabytes de datos y meses de cómputo. Está fuera del alcance de cualquier proyecto académico de pregrado. Los Niveles 2 y 3 representan el reto apropiado para una asignatura de ingeniería.

1.1 Comparativa de Niveles

Criterio	Nivel 2: Fine-Tuning	Nivel 3: Desde Cero	Combinado (Recomendado)
Tiempo estimado	1–2 semanas	2–3 semanas	3–4 semanas
Hardware mínimo	GPU 8GB (Colab gratuito)	CPU suficiente para dataset pequeño	GPU 8GB
Modelo resultante	Útil para producción	Educativo (no productivo)	Ambos beneficios
Complejidad teórica	Media	Alta	Alta
Complejidad práctica	Alta	Media-Alta	Alta
Valor académico	Alto	Muy alto	Máximo

NIVEL 2: FINE-TUNING

2. Fine-Tuning de LLMs Preentrenados

El fine-tuning consiste en tomar un modelo de lenguaje ya entrenado en grandes cantidades de texto general (como Llama 3, Mistral o Phi-3) y continuar su entrenamiento con un dataset especializado, ajustando sus pesos para que el modelo se desempeñe mejor en una tarea o dominio específico. En el contexto del proyecto, esto significa adaptar un LLM para que entienda y extraiga información de papers científicos con mayor precisión.

2.1 Conceptos Fundamentales

2.1.1 Transfer Learning en LLMs

Un LLM preentrenado ya ha aprendido representaciones ricas del lenguaje natural a partir de cientos de gigabytes de texto. El fine-tuning aprovecha este conocimiento previo y lo redirige hacia una tarea específica, requiriendo muchos menos datos y cómputo que entrenar desde cero.

2.1.2 Tipos de Fine-Tuning

Tipo	Descripción	Cuándo usarlo
Full Fine-Tuning	Actualiza todos los pesos del modelo. Máxima adaptación pero costoso en memoria.	Dataset grande (>100K ejemplos), GPU >80GB
LoRA (Low-Rank Adaptation)	Agrega matrices de rango bajo entrenables, congela los pesos originales. El estándar actual.	Dataset mediano, GPU 8–24GB
QLoRA	LoRA con cuantización 4-bit. Permite fine-tuning en GPUs de consumo (incluso Colab gratuito).	Dataset pequeño/mediano, GPU 4–8GB
Instruction Tuning	Fine-tuning con pares instrucción-respuesta para mejorar seguimiento de instrucciones.	Adaptar el modelo a seguir prompts del agente
RLHF / DPO	Refuerzo con retroalimentación humana / Direct Preference Optimization para alinear con preferencias.	Mejora de calidad de respuestas generadas

2.2 Papers Fundamentales de Fine-Tuning

2.2.1 LoRA — El método estándar de la industria

LoRA: Low-Rank Adaptation of Large Language Models

Autores: Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, et al.
Venue: ICLR 2022
ArXiv: https://arxiv.org/abs/2106.09685

PROBLEMA: El full fine-tuning de un LLM de 175B parámetros requiere actualizar todos sus pesos, lo cual es computacionalmente prohibitivo. LoRA demuestra que los cambios de pesos durante la adaptación tienen un rango intrínseco bajo.

MÉTODO: En lugar de actualizar directamente la matriz de pesos W de dimensión d×d, LoRA agrega dos matrices entrenables A (d×r) y B (r×d) donde r << d. El forward pass se convierte en: h = Wx + BAx. Solo A y B se entrenan, reduciendo parámetros entrenables en un factor de hasta 10,000x.

RESULTADOS: Fine-tuning de GPT-3 con LoRA alcanza calidad comparable al full fine-tuning en múltiples benchmarks, con 3x menos VRAM y sin overhead de inferencia.

RELEVANCIA PARA EL PROYECTO: LoRA permite fine-tunear Llama 3 8B en un dataset de papers científicos usando Google Colab gratuito.

2.2.2 QLoRA — Fine-tuning en hardware de consumo

QLoRA: Efficient Finetuning of Quantized LLMs

Autores: Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, Luke Zettlemoyer
Venue: NeurIPS 2023
ArXiv: https://arxiv.org/abs/2305.14314

PROBLEMA: Incluso LoRA requiere cargar el modelo base en precisión completa (FP16), lo que consume 16GB+ de VRAM para modelos de 7B parámetros.

MÉTODO: QLoRA cuantiza el modelo base a 4 bits (NF4 - Normal Float 4), reduciendo el uso de memoria a ~6GB para modelos de 7B. Introduce Double Quantization (cuantizar los factores de cuantización) y paginación de optimizadores para manejar gradientes en picos de memoria.

RESULTADOS: Fine-tuning de Llama 65B en una sola GPU A100 de 48GB, con calidad comparable al full fine-tuning. Guanaco 65B entrenado con QLoRA alcanza el 99.3% del rendimiento de ChatGPT.

RELEVANCIA: Hace posible el fine-tuning en Google Colab T4 (16GB VRAM) de modelos de hasta 13B parámetros.

2.2.3 Instruction Tuning — Adaptar el modelo a seguir instrucciones

FLAN: Finetuned Language Models Are Zero-Shot Learners

Autores: Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, et al. (Google)
Venue: ICLR 2022
ArXiv: https://arxiv.org/abs/2109.01652

PROBLEMA: Los LLMs preentrenados no siguen instrucciones de forma confiable sin ejemplos en el prompt.

MÉTODO: Fine-tuning con más de 60 datasets de NLP reformateados como pares instrucción-respuesta. El modelo aprende a interpretar instrucciones en lenguaje natural de forma generalizable a tareas no vistas.

RELEVANCIA: Fundamento teórico para crear el dataset de instruction tuning del agente de revisión de literatura. Cada par del dataset tendría la forma: instrucción='Extrae el método principal de este paper' → respuesta='{"metodologia": "..."}'.

2.2.4 DPO — Alineamiento sin entrenador de recompensa

Direct Preference Optimization (DPO)

Autores: Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, et al.
Venue: NeurIPS 2023
ArXiv: https://arxiv.org/abs/2305.18290

PROBLEMA: RLHF (Reinforcement Learning from Human Feedback) requiere entrenar un modelo de recompensa separado y usar PPO, proceso inestable y computacionalmente costoso.

MÉTODO: DPO reformula el problema como clasificación binaria directa sobre pares de preferencias (respuesta preferida vs. rechazada), eliminando el modelo de recompensa y el loop de RL. Una sola pérdida derivada de la política óptima.

RELEVANCIA: Permite mejorar la calidad de los reportes generados por el agente usando un dataset de preferencias simples: dado un paper, ¿qué reporte es mejor, A o B? Sin necesidad de RLHF complejo.

2.2.5 Alpaca — Instruction Tuning económico como referencia

Alpaca: A Strong, Replicable Instruction-Following Model

Autores: Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, et al. (Stanford CRFM)
Venue: Blog Stanford / Technical Report 2023
Link: https://crfm.stanford.edu/2023/03/13/alpaca.html

PROBLEMA: Demostrar que instruction tuning de alta calidad puede lograrse con recursos limitados.

MÉTODO: Fine-tuning de LLaMA 7B con 52,000 ejemplos instrucción-respuesta generados por GPT-3.5 (self-instruct). Costo total del proceso: menos de 600 dólares.

RESULTADOS: Alpaca 7B alcanza calidad comparable a text-davinci-003 en muchas tareas de instrucción.

RELEVANCIA: Demuestra que construir un dataset de instrucciones para el agente (usando GPT-4o para generar pares sintéticos de extracción de papers) es viable y económico.

2.3 Modelos Base Recomendados

Los siguientes modelos open-source son los más adecuados para fine-tuning en el proyecto, ordenados por accesibilidad:

Modelo	Parámetros	VRAM mínima	Licencia	Mejor para
Phi-3.5-mini	3.8B	4GB (QLoRA)	MIT	Recursos muy limitados, Colab gratuito
Llama 3.2 3B	3B	4GB (QLoRA)	Llama 3.2 License	Balance calidad/velocidad, recomendado para empezar
Llama 3.1 8B	8B	8GB (QLoRA)	Llama 3.1 License	Calidad alta, Colab Pro o GPU local 8GB
Mistral 7B v0.3	7B	8GB (QLoRA)	Apache 2.0	Más permisivo que Llama, bueno para papers en inglés
Qwen2.5 7B	7B	8GB (QLoRA)	Apache 2.0	Excelente rendimiento en extracción de información estructurada

2.4 Repositorios GitHub de Referencia

2.4.1 Repos principales para implementación

Repositorio	URL	Para qué sirve
huggingface/trl	github.com/huggingface/trl	Librería oficial para SFT, DPO, PPO. El estándar del campo. SFTTrainer es el punto de entrada más simple.
artidoro/qlora	github.com/artidoro/qlora	Implementación original de QLoRA de los autores del paper. Incluye script de entrenamiento completo qlora.py.
huggingface/peft	github.com/huggingface/peft	Parameter-Efficient Fine-Tuning: LoRA, QLoRA, IA3, AdaLoRA. La forma más simple de aplicar LoRA con 5 líneas.
unsloth/unsloth	github.com/unslothai/unsloth	Fine-tuning 2x más rápido y 80% menos memoria que Hugging Face estándar. Kernels CUDA optimizados. Ideal para Colab.

Repositorio	URL	Para qué sirve
tatsu-lab/stanford_alpaca	github.com/tatsu-lab/stanford_alpaca	Código completo del proyecto Alpaca: generación de datos, fine-tuning, evaluación. Referencia directa para replicar.
microsoft/DeepSpeedExamples	github.com/microsoft/DeepSpeedExamples	Ejemplos de fine-tuning distribuido con DeepSpeed. Para modelos grandes en múltiples GPUs.

2.5 Roadmap Técnico: Fine-Tuning Paso a Paso

Fase 1 — Preparar el dataset de entrenamiento

El dataset es el componente más crítico. Para el agente de revisión de literatura, necesitas pares (input, output) donde:

- Input: fragmento de un paper científico (sección de métodos, resultados, etc.)
- Output: JSON estructurado con {problema, metodologia, datasets, resultados, contribuciones}

Estrategia de generación de datos sintéticos (Alpaca-style):

```
# 1. Recolectar 500-1000 papers de ArXiv en tu dominio
# 2. Usar GPT-4o-mini para generar los pares de extracción
# Costo estimado: ~$5-10 USD para 1000 ejemplos

prompt = '''Dado este fragmento de un paper científico, genera un JSON
con los campos: problema, metodologia, datasets, resultados, limitaciones.
Si algún campo no aplica, usa null.

FRAGMENTO: {chunk}

JSON:'''
```

Fase 2 — Configurar el entrenamiento con QLoRA

```
# pip install unsloth transformers datasets trl peft bitsandbytes

from unsloth import FastLanguageModel
import torch

# Cargar modelo base con cuantización 4-bit
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name='unsloth/Llama-3.2-3B-Instruct',
    max_seq_length=2048,
    load_in_4bit=True,  # QLoRA: 4-bit quantization
)

# Configurar LoRA
model = FastLanguageModel.get_peft_model(
    model,
```

```
    r=16,           # Rango de las matrices LoRA
    lora_alpha=16,  # Factor de escala
    target_modules=['q_proj', 'k_proj', 'v_proj', 'o_proj'],
    lora_dropout=0.05,
    bias='none',
)
```

Fase 3 — Entrenar con SFTTrainer (Supervised Fine-Tuning)

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    dataset_text_field='text',
    max_seq_length=2048,
    args=TrainingArguments(
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        num_train_epochs=3,
        learning_rate=2e-4,
        output_dir='./outputs',
        fp16=True,
    ),
)
trainer.train()

# Guardar adaptador LoRA
model.save_pretrained('./lora_adapter')
```

Fase 4 — Evaluar el modelo fine-tuneado

Métricas de evaluación recomendadas para el agente de revisión de literatura:

- ROUGE-L: similitud de secuencias entre extracción generada y referencia
- BERTScore: similitud semántica usando embeddings contextuales
- F1 por campo: qué porcentaje de campos del JSON se extraen correctamente
- Human Evaluation: comparar reporte generado vs. resumen humano del mismo paper

2.6 Recursos Adicionales de Fine-Tuning

Recurso	Descripción y URL
Hugging Face Fine-tuning Docs	Documentación oficial completa: huggingface.co/docs/transformers/training
Unsloth Notebooks	Notebooks de Colab listos para ejecutar: github.com/unslothai/unsloth#-finetune-for-free
LLM Course (Hugging Face)	Curso gratuito completo de 3 módulos: huggingface.co/learn/llm-course

Recurso	Descripción y URL
Paper: Self-Instruct	Generar datos de entrenamiento desde el propio LLM (arxiv.org/abs/2212.10560)
Paper: Scaling Laws	Cuántos datos y parámetros necesitas (arxiv.org/abs/2001.08361)
WandB Integration	Visualizar métricas de entrenamiento en tiempo real: wandb.ai/site/articles/llm-finetuning

NIVEL 3: TRANSFORMER DESDE CERO

3. Implementar el Transformer desde Cero en PyTorch

Implementar la arquitectura Transformer completa en PyTorch sin usar abstracciones de alto nivel (sin transformers, sin LangChain) es el ejercicio que mejor demuestra comprensión del funcionamiento interno de los LLMs. El modelo resultante no va a ser capaz de leer papers en producción, pero el proceso de construirlo desde cero establece los fundamentos teóricos del proyecto.

OBJETIVO	Implementar un GPT pequeño (caracter-level o token-level con BPE básico) entrenado en abstracts de papers científicos. La capacidad del modelo no es el objetivo: demostrar que se entiende cada componente de la arquitectura lo es.

3.1 La Arquitectura Transformer

El Transformer, introducido en 2017, reemplazó a las RNNs y LSTMs como arquitectura dominante en NLP. Su mecanismo central —la atención multi-cabeza— permite al modelo relacionar cualquier par de posiciones en la secuencia de entrada sin importar su distancia, superando el problema de dependencias de largo plazo.

3.1.1 Componentes de la arquitectura

Componente	Descripción técnica detallada
Token Embeddings	Matriz E de dimensión V×d_model que mapea cada token a un vector denso. Los pesos se aprenden durante el entrenamiento. V = vocabulario, d_model = dimensión del modelo (típicamente 512 o 768).
Positional Encoding	Representación sinusoidal que añade información de posición: PE(pos,2i) = sin(pos/10000^(2i/d)). Permite al modelo conocer el orden de los tokens sin estructura recurrente.
Multi-Head Attention (MHA)	El mecanismo central. Proyecta Q, K, V h veces con matrices Wq, Wk, Wv diferentes (heads). Cada head aprende a atender a distintos tipos de relaciones. Las salidas se concatenan y proyectan con Wo.
Scaled Dot-Product Attention	Attn(Q,K,V) = softmax(QK^T / sqrt(d_k)) * V. La división por sqrt(d_k) estabiliza los gradientes cuando d_k es grande. El resultado es una combinación ponderada de valores V donde los pesos reflejan similaridad Q-K.
Feed-Forward Network (FFN)	Dos capas lineales con activación GELU/ReLU: FFN(x) = max(0, xW1 + b1)W2 + b2. d_ff típicamente 4× d_model. Procesa cada posición independientemente.
Layer Normalization	LayerNorm(x) = gamma * (x - mean) / std + beta. Normaliza sobre la dimensión del embedding, no sobre el batch. Estabiliza el entrenamiento profundo.

Componente	Descripción técnica detallada
Residual Connections	x + Sublayer(x). Permiten el flujo de gradientes a través de capas profundas (hasta 96 capas en GPT-3). Esenciales para entrenar Transformers profundos.
Causal Masking (Decoder)	Máscara triangular inferior que impide que la posición i atienda posiciones j > i. Garantiza que la generación sea autoregresiva: cada token solo depende del contexto pasado.

3.2 Papers Fundamentales

3.2.1 El paper original — Attention Is All You Need

Attention Is All You Need

Autores: Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, et al. (Google Brain)
Venue: NeurIPS 2017
ArXiv: https://arxiv.org/abs/1706.03762
Citas: >100,000 — uno de los papers más citados en la historia del machine learning

PROBLEMA: Las RNNs procesan secuencias de forma secuencial, lo que limita la paralelización y dificulta capturar dependencias de largo alcance.

CONTRIBUCIÓN: Elimina la recurrencia y la convolución completamente, basando toda la arquitectura en mecanismos de atención. El Transformer encoder-decoder original fue diseñado para traducción automática.

QUÉ IMPLEMENTAR DEL PAPER: Sección 3 completa (The Model Architecture). Especialmente las ecuaciones 1 (Scaled Dot-Product Attention) y 2 (Multi-Head Attention). Son las 10 páginas más influyentes del NLP moderno.

NOTA: Para GPT (generativo) solo se usa el decoder del Transformer. Para BERT (representaciones) solo el encoder.

3.2.2 GPT — El paradigma autoregresivo

GPT-1: Improving Language Understanding by Generative Pre-Training

Autores: Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever (OpenAI)
Venue: OpenAI Technical Report 2018
Link: https://openai.com/research/language-unsupervised

CONTRIBUCIÓN: Adapta el decoder del Transformer para pre-entrenamiento no supervisado en grandes corpora de texto, seguido de fine-tuning supervisado. El paradigma GPT (Generative Pre-trained Transformer) que domina el campo.

DIFERENCIAS CON EL TRANSFORMER ORIGINAL: Usa solo el decoder, elimina el cross-attention (no hay encoder), usa causal masking en self-attention. Arquitectura de 12 capas, 768 dimensiones.

RELEVANCIA: Tu implementación desde cero reproduce esencialmente la arquitectura GPT-1/2. El paper muestra exactamente qué hiperparámetros usar para un modelo de tamaño educational.

3.2.3 GPT-2 — Escalado y generación de texto

GPT-2: Language Models are Unsupervised Multitask Learners

Autores: Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever (OpenAI)
Venue: OpenAI Technical Report 2019
Link: https://openai.com/research/better-language-models

CONTRIBUCIÓN: Demuestra que escalar el modelo y los datos (1.5B parámetros, WebText de 40GB) produce un modelo capaz de realizar múltiples tareas sin fine-tuning explícito. Introduce Pre-Norm (LayerNorm antes del sub-layer, no después).

CAMBIO ARQUITECTÓNICO IMPORTANTE: GPT-2 mueve LayerNorm al inicio de cada bloque (Pre-Norm) en lugar de después (Post-Norm del Transformer original). Esto mejora significativamente la estabilidad del entrenamiento en modelos profundos.

RELEVANCIA: La implementación educativa nanoGPT de Karpathy reproduce GPT-2 exactamente. Los pesos de GPT-2 Small (124M) están disponibles para verificar tu implementación.

3.2.4 BERT — Encoders bidireccionales

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Autores: Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova (Google AI)
Venue: NAACL 2019
ArXiv: https://arxiv.org/abs/1810.04805

CONTRIBUCIÓN: Usa el encoder del Transformer con atención bidireccional (ve todo el contexto). Pre-entrenado con Masked Language Modeling (MLM) y Next Sentence Prediction. Revolución en tareas de comprensión de texto.

DIFERENCIA CLAVE CON GPT: BERT es bidireccional (ve contexto izquierdo Y derecho) pero no puede generar texto autoregresivamente. GPT es unidireccional pero puede generar.

RELEVANCIA: Si el objetivo es extracción de información (clasificación de tokens) en lugar de generación, BERT/RoBERTa podría ser más apropiado que GPT para el agente.

3.2.5 Scaling Laws — Cuántos parámetros y datos necesitas

Scaling Laws for Neural Language Models

Autores: Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, et al. (OpenAI)
Venue: arXiv 2020
ArXiv: https://arxiv.org/abs/2001.08361

CONTRIBUCIÓN: Establece relaciones de ley de potencias entre el rendimiento del LLM y tres factores: número de parámetros N, tamaño del dataset D, y cómputo de entrenamiento C. El rendimiento mejora suavemente y de forma predecible al escalar.

RESULTADO PRÁCTICO: La relación óptima parámetros/tokens de entrenamiento es aproximadamente N = 20D (20 tokens por parámetro). Para un modelo educational de 1M parámetros, necesitas ~20M tokens.

RELEVANCIA: Te dice exactamente qué tamaño de modelo construir y cuántos datos de abstracts de papers necesitas para que el entrenamiento tenga sentido.

3.3 Repositorios GitHub de Referencia

3.3.1 El recurso más importante: nanoGPT

★ EMPEZAR AQUÍ	github.com/karpathy/nanoGPT — GPT-2 implementado desde cero en ~300 líneas de Python/PyTorch por Andrej Karpathy (ex-Director de IA de Tesla, cofundador de OpenAI). El mejor recurso educativo existente para implementar un Transformer.

nanoGPT incluye:

- model.py: toda la arquitectura GPT en ~300 líneas, bien comentada
- train.py: loop de entrenamiento completo con gradient accumulation y checkpointing
- data/shakespeare/prepare.py: ejemplo de cómo preparar datos de texto
- config/: configuraciones de hiperparámetros para distintos tamaños de modelo

Video complementario obligatorio: "Let's build GPT: from scratch, in code, spelled out" (Andrej Karpathy, 2023). Duración: 2 horas. URL: https://www.youtube.com/watch?v=kCc8FmEb1nY. Construye nanoGPT línea por línea explicando cada decisión.

Repositorio	URL	Qué muestra
karpathy/nanoGPT	github.com/karpathy/nanoGPT	GPT-2 completo en 300 líneas. EL punto de partida.
karpathy/minGPT	github.com/karpathy/minGPT	Versión anterior más modular. Buena para entender la estructura de clases.
sebastianraschka/LLMs-from-scratch	github.com/rasbt/LLMs-from-scratch	Libro completo 'Build a Large Language Model from Scratch' con código. Muy detallado, nivel perfecto para pregrado.
jaymody/picoGPT	github.com/jaymody/picoGPT	GPT-2 implementado en 60 líneas de NumPy puro. Para entender la matemática sin abstracciones de PyTorch.
karpathy/makemore	github.com/karpathy/makemore	Series de videos y código construyendo desde bigrams hasta Transformers. Curva de aprendizaje perfecta.
harvardnlp/annotated-transformer	github.com/harvardnlp/annotated-transformer	El paper 'Attention Is All You Need' implementado línea por línea con anotaciones. Referencia académica.
the-ai-guy/bert-from-scratch	github.com/the-ai-guy/bert-from-scratch	BERT implementado desde cero. Para quienes prefieran el enfoque encoder+extracción de info.

3.4 Roadmap Técnico: Transformer desde Cero

Fase 1 — Matemáticas fundamentales (2-3 días)

Antes de implementar, asegúrate de entender:

- Multiplicación matricial y producto punto
- Softmax: cómo convierte logits en distribución de probabilidad
- Backpropagation: cómo fluyen los gradientes
- Embeddings: qué es un vector de representación densa

Recurso: Serie de videos "Neural Networks: Zero to Hero" de Andrej Karpathy en YouTube. Los primeros 4 videos (~8 horas en total) cubren exactamente estos fundamentos.

Fase 2 — Implementar Scaled Dot-Product Attention

Esta es la pieza más importante. Empezar aquí:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

def scaled_dot_product_attention(Q, K, V, mask=None):
    d_k = Q.size(-1)
    # Scores: (batch, heads, seq_len, seq_len)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(d_k)

    if mask is not None:  # Causal mask para GPT
        scores = scores.masked_fill(mask == 0, float('-inf'))

    attn_weights = F.softmax(scores, dim=-1)
    return torch.matmul(attn_weights, V), attn_weights
```

Fase 3 — Multi-Head Attention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.num_heads = num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        B, T, C = x.shape
        Q = self.W_q(x).view(B, T, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(B, T, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(B, T, self.num_heads, self.d_k).transpose(1, 2)
        out, _ = scaled_dot_product_attention(Q, K, V, mask)
        out = out.transpose(1, 2).contiguous().view(B, T, C)
        return self.W_o(out)
```

Fase 4 — Bloque Transformer completo

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.attn = MultiHeadAttention(d_model, num_heads)
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model),
            nn.Dropout(dropout)
        )
        self.ln1 = nn.LayerNorm(d_model)  # Pre-Norm (estilo GPT-2)
        self.ln2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # Pre-Norm + Residual
        x = x + self.dropout(self.attn(self.ln1(x), mask))
        x = x + self.dropout(self.ff(self.ln2(x)))
        return x
```

Fase 5 — Modelo GPT completo

```
class MiniGPT(nn.Module):
    def __init__(self, vocab_size, d_model=256, num_heads=8,
                 num_layers=6, d_ff=1024, max_len=512, dropout=0.1):
        super().__init__()
        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = nn.Embedding(max_len, d_model)
        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, num_heads, d_ff, dropout)
            for _ in range(num_layers)
        ])
        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size, bias=False)
        self.max_len = max_len

    def forward(self, idx):
        B, T = idx.shape
        tok = self.token_emb(idx)  # (B, T, d_model)
        pos = self.pos_emb(torch.arange(T, device=idx.device))
        x = tok + pos
        mask = torch.tril(torch.ones(T, T, device=idx.device))
        for block in self.blocks:
            x = block(x, mask)
        return self.head(self.ln_f(x))
```

Fase 6 — Dataset de abstracts de papers

Entrena el modelo en abstracts de papers de ArXiv relacionados con el tema del proyecto:

```
import arxiv
```

```
client = arxiv.Client()
search = arxiv.Search(
    query='large language model NLP',
    max_results=5000,
    sort_by=arxiv.SortCriterion.SubmittedDate
)

texts = []
for paper in client.results(search):
    if paper.summary:  # abstract
        texts.append(paper.summary)

# Guardar corpus
with open('abstracts.txt', 'w') as f:
    f.write('\n\n'.join(texts))

# Resultado: ~5000 abstracts = ~5M tokens aproximadamente
# Suficiente para un modelo educational de 1-5M parámetros
```

3.5 Hiperparámetros Recomendados según Capacidad de Hardware

Hardware	Parámetros aprox.	d_model	Layers	Heads	d_ff	Tiempo entreno
CPU (cualquiera)	~500K	128	4	4	512	~2h (10K steps)
Colab T4 gratis	~3M	256	6	8	1024	~1h (50K steps)
Colab A100 Pro	~25M	512	8	8	2048	~2h (100K steps)
GPU local 8GB	~85M	768	12	12	3072	~4h (200K steps)

3.6 Recursos Adicionales del Nivel 3

Recurso	Descripción y URL
The Illustrated Transformer	Explicación visual del mecanismo de atención: jalammar.github.io/illustrated-transformer/
The Annotated Transformer	El paper original implementado con anotaciones detalladas: nlp.seas.harvard.edu/annotated-transformer/
PyTorch Tutorial Oficial	Tutorial de sequence-to-sequence con Transformer: pytorch.org/tutorials/beginner/transformer_tutorial.html
Paper: Flash Attention 2	Atención eficiente en memoria (arxiv.org/abs/2307.08691). Para acelerar el entrenamiento si tienes GPU.

Recurso	Descripción y URL
Paper: Rotary Embeddings (RoPE)	Embeddings posicionales usados en Llama/GPT-4 (arxiv.org/abs/2104.09864). Alternativa a positional encoding sinusoidal.
LLM Visualization (interactivo)	Visualización 3D interactiva del Transformer: bbycroft.net/llm
Paper: RoBERTa	BERT entrenado correctamente: mejores prácticas de pre-entrenamiento (arxiv.org/abs/1907.11692)

4. Estrategia Combinada Recomendada para el Proyecto

La estrategia óptima para maximizar el valor académico del proyecto de Agentes Inteligentes combina ambos niveles: implementar el Transformer desde cero para demostrar comprensión de los fundamentos (Nivel 3) y usar fine-tuning de un modelo preentrenado para el agente funcional que realmente extrae información de papers (Nivel 2).

4.1 División del Trabajo

Semana	Nivel	Entregable	Papers/Recursos clave
1	3 — Fundamentos	Atención y bloque Transformer implementados y testeados	Vaswani et al. 2017 (Attn Is All You Need)
2	3 — MiniGPT	MiniGPT entrenado en abstracts de ArXiv. Puede generar texto coherente.	nanoGPT (Karpathy), Scaling Laws (Kaplan et al.)
3	2 — Fine-tuning	Dataset de extracción de info generado. Fine-tuning de Llama 3.2 3B con QLoRA.	QLoRA (Dettmers et al.), Alpaca (Taori et al.)
4	Integración	Agente completo: MiniGPT + LLM fine-tuneado + reporte evaluado	FLAN (Wei et al.), DPO (Rafailov et al.)

4.2 Estructura del Reporte Final del Proyecto

El informe académico del proyecto debe justificar las decisiones técnicas haciendo referencia explícita a los papers. Una estructura sugerida:

1. Introducción: problema de revisión de literatura, solución propuesta
2. Marco teórico: arquitectura Transformer (Vaswani et al.), ReAct (Yao et al.), RAG (Lewis et al.)
3. Nivel 3 — Implementación desde cero: decisiones de diseño, hiperparámetros, resultados de entrenamiento
4. Nivel 2 — Fine-tuning: dataset generado, configuración QLoRA/LoRA, métricas de evaluación
5. Integración: cómo el LLM fine-tuneado potencia el agente de revisión
6. Evaluación: ROUGE-L, BERTScore, evaluación humana de reportes generados
7. Conclusiones y trabajo futuro

4.3 Lista Completa de Papers a Citar

Los siguientes papers deben aparecer en las referencias del proyecto, agrupados por componente:

Arquitectura del agente:

- ReAct — Yao et al. (2023). arxiv.org/abs/2210.03629
- CoALA — Sumers et al. (2023). arxiv.org/abs/2309.02427
- RAG — Lewis et al. (2020). arxiv.org/abs/2005.11401
- Reflexion — Shinn et al. (2023). arxiv.org/abs/2303.11366

Transformer y LLMs (Nivel 3):

- Attention Is All You Need — Vaswani et al. (2017). arxiv.org/abs/1706.03762
- GPT-1 — Radford et al. (2018). openai.com/research/language-unsupervised
- GPT-2 — Radford et al. (2019). openai.com/research/better-language-models
- BERT — Devlin et al. (2019). arxiv.org/abs/1810.04805
- Scaling Laws — Kaplan et al. (2020). arxiv.org/abs/2001.08361

Fine-Tuning (Nivel 2):

- LoRA — Hu et al. (2022). arxiv.org/abs/2106.09685
- QLoRA — Dettmers et al. (2023). arxiv.org/abs/2305.14314
- FLAN — Wei et al. (2022). arxiv.org/abs/2109.01652
- Alpaca — Taori et al. (2023). crfm.stanford.edu/2023/03/13/alpaca.html
- DPO — Rafailov et al. (2023). arxiv.org/abs/2305.18290

Extracción de información científica:

- SciREX — Jain et al. (2020). arxiv.org/abs/2005.00512
- Chain-of-Thought — Wei et al. (2022). arxiv.org/abs/2201.11903

Resumen de Repositorios GitHub por Componente

Nivel	Repositorio	URL y propósito
3 - Core	karpathy/nanoGPT	github.com/karpathy/nanoGPT — GPT-2 en 300 líneas, punto de partida obligatorio
3 - Core	karpathy/minGPT	github.com/karpathy/minGPT — versión más modular, mejor para estudiar estructura
3 - Libro	rasbt/LLMs-from-scratch	github.com/rasbt/LLMs-from-scratch — LLM completo desde cero con libro de referencia
3 - Minimalista	jaymody/picoGPT	github.com/jaymody/picoGPT — GPT-2 en 60 líneas NumPy, para entender la matemática
3 - Referencia	harvardnlp/annotated-transformer	github.com/harvardnlp/annotated-transformer — paper original implementado
2 - Core	huggingface/trl	github.com/huggingface/trl — SFT, DPO, PPO. La librería estándar de fine-tuning
2 - Core	huggingface/peft	github.com/huggingface/peft — LoRA, QLoRA y otras técnicas PEFT en 5 líneas
2 - Velocidad	unslothai/unsloth	github.com/unslothai/unsloth — fine-tuning 2x más rápido, ideal para Colab
2 - Referencia	artidoro/qlora	github.com/artidoro/qlora — implementación original de QLoRA de los autores
2 - Referencia	tatsu-lab/stanford_alpaca	github.com/tatsu-lab/stanford_alpaca — pipeline completo de instruction tuning

Documento generado como guía técnica para el proyecto de la asignatura Agentes Inteligentes — 2025